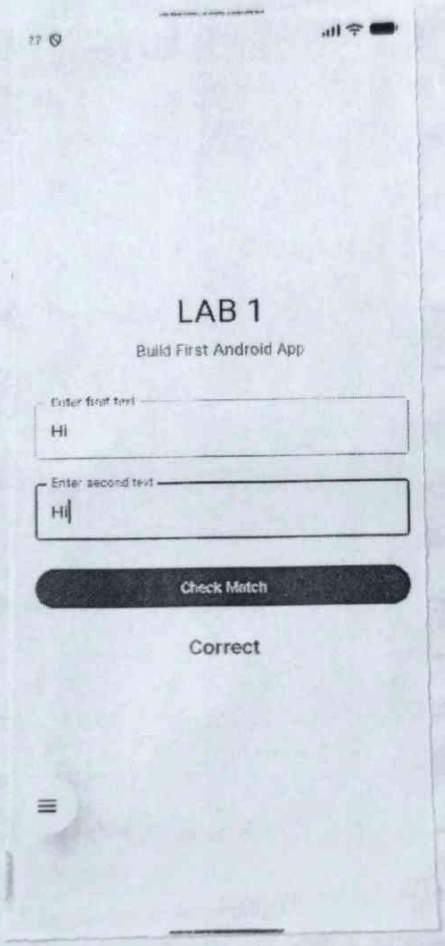
INDEX

NAME: Manishaa. G STD.: CSE SEC.: ROLL NO.: 23070116 SUB.: MAD LAB.

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.		Drawing Shapes on canvas using custom view		
2.		Build First Android App - Text Match Validation		
3.		App Navigation - Activity Transition b/w multiple screen		
4.		Activity & Fragment Lifecycle Simple Timer with Lifecycle event.		
5.		App Architecture (UI layer)		
6		App Architecture (persistence)		
7.		Advanced recycle view use case		
8.		Connect to Internet		
9.		Repository pattern and Work Manager.		
10.		App UI Design.		


LAB 1
Build First Android App
Enter first text
Hi
Enter second text
Hi
Check Match
Correct

Exp. No : 1

Drawing Shapes on Canvas using custom view in Android

Aim: To create an android application demonstrating custom drawing using canvas with interactive shape controls to understand how to render 2D graphics programmatically in Android.

Algorithm:

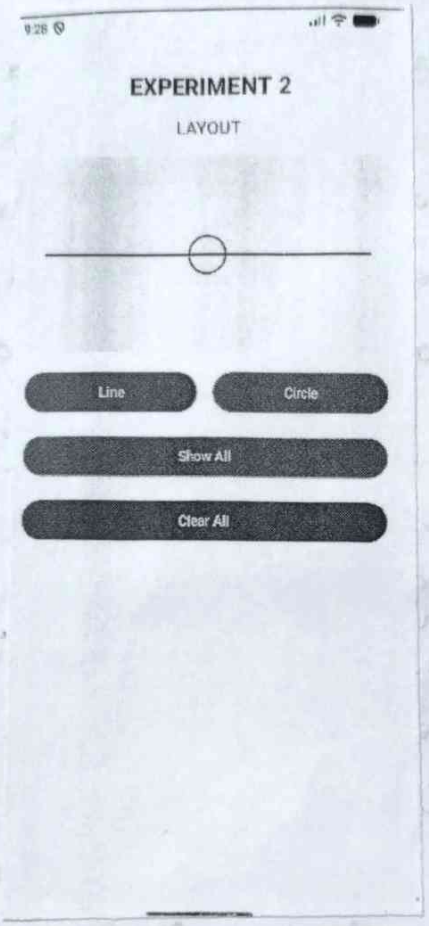
1. Open Android Studio and create a new android project.
2. Name the project and set the package name as com.ex.lab1
3. Create a new kotlin class that extends view and override the onDraw() method and create a paint object
4. In the main layout, XML file, add the custom view along with four buttons - line, circle, showAll, clearAll
5. In Main Activity, get reference to all buttons and custom view using findviewbyId
6. Maintain boolean flags to track visible shapes
7. Set onclickListener for each button to update the flags.
8. Call invalidate() on the custom view after each button click to trigger a redraw.
9. In OnDraw(), check flags and draw only the shapes that are currently active
10. Run the app on emulator and verify each button working

Output:

The application displays a white canvas area at the top of the screen. A horizontal blue line and a red circle are drawn on the canvas. Below the canvas, four buttons are displayed.

Result:

The android application was successfully created and executed.


9:28
EXPERIMENT 2
LAYOUT
Line
Circle
Show All
Clear All

Ex. No : 2 | Build First Android App - Text Match Validation using Jetpack compose.

Aim : To build an Android application using Jetpack compose that accepts two text inputs from user and validation whether the entered strings match each other and demonstrate use of state management.

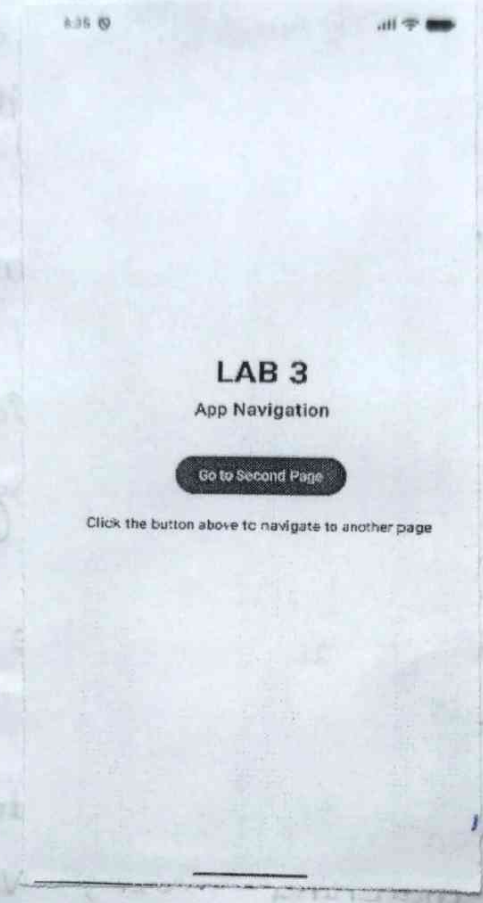
Algorithm :

1. Open Android studio and create a new android project with the empty compose activity template.
2. Name the project and set package name as com.ex.lab2
3. Open Main Activity.kt and set up the compose content.
4. Create a composable function.
5. Inside validation App, declare two mutable state variable.
6. Declare another mutable state Variable.
7. Use a column composable to arrange UI elements.
8. Add a Button composable labeled Check match.
9. Add a text composable below the button to display the result message.
10. Build and run app on an emulator, enter matching or non-matching text, verify the output.

Output :

The application displays a screen titled LAB2. with the subtitle build Android.app.

The Android Application using Jetpack compose was successfully created and executed on the emulator.


LAB 3
App Navigation
Go to Second Page
Click the button above to navigate to another page

Ex. No : 3 | App Navigation - Activity Transition between multiple screens in android.

Aim:

To demonstrate multi-screen navigation in an android application by implementing bi-directional transitions between two activities using android intents, helping to understand the concept of Activity stack and navigation flow.

Algorithm:

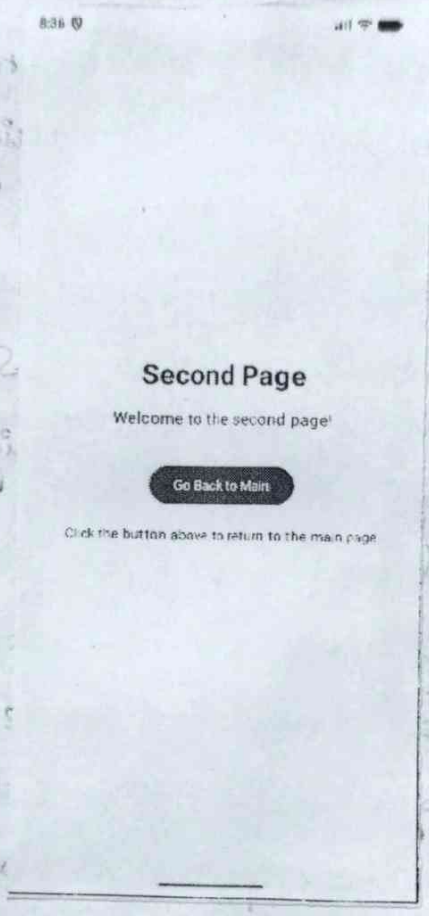
1. Open Android Studio and Create a new project & name it
2. Open activity_main.xml and design the main layout using Jetpack compose.
3. Add a title Text Composable.
4. Add a button labeled - Go to Second Page.
5. Create a new kotlin file and design the second screen layout.
6. Add a button labled - go back.
7. In main activity, set an onclick Listener.
8. Inside the click handler, create an Intent object
9. Call StartActivity(), set an onclick Listener.
10. Build and run the application on the emulator and test navigation in both direction.

Output:

The application displays the title LAB 8 and subtitle activity and fragment lifecycle at the top

Result:

The android application was successfully built and executed.


8:36
Second Page
Welcome to the second page!
Go Back to Main
Click the button above to return to the main page

Ex-No : 4 | Activity and Fragment Lifecycles - Simple Timer with Lifecycle event Logging.

Aim: To understand the android activity and Fragment lifecycles callback event with stamps, demonstrating how the system manages activity states.

Algorithm:

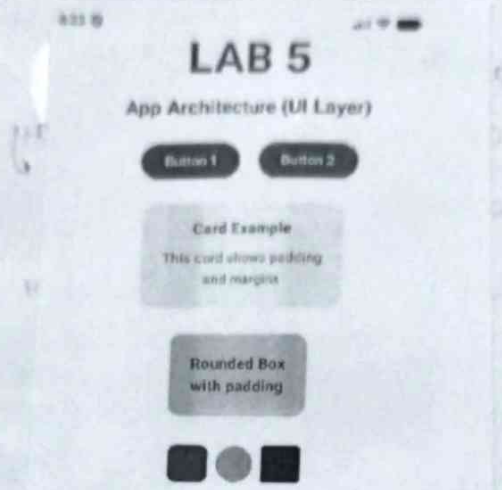
1. Open Android Studio and create a new Android project using empty activity.
2. Name the project and set the package name as com.ex.Lab4.
3. Open activity_main.xml.
4. Add a Textview at the top to display the app title.
5. Add another Textview.
6. Add a textview to display timer value and add three buttons.
7. Declare Variables
8. In the Start button click listener.
9. In pause button click listener.
10. In Stop button click listener.
11. Build and run the app on the emulator, interact with the timer buttons and observe the lifecycle logs both on screen and in logcat.

Output:

The application displays the title LAB4 and subtitle activity and Fragment Lifecycle at the top.

Result:

The android application was succesfully built and executed.


LAB 5
App Architecture (UI Layer)
Button 1
Button 2
Card Example
This card shows padding and margins
Rounded Box with padding

Ex. No : 5

App Architecture (UI Layer)

Aim:

To understand the UI Layer in Android app architecture, which is responsible for displaying data on the screen and handling user interactions using activities, fragments and view components.

Algorithm:

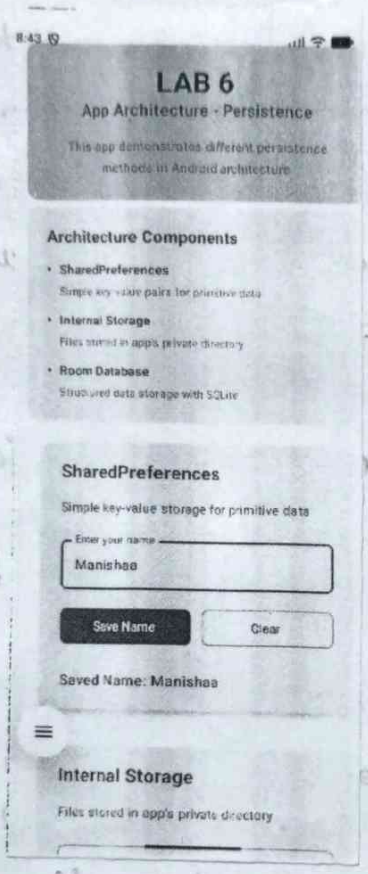
1. Open Android Studio and create new project using Empty Activity.
2. Name the project and set the package name as com.ex.lab
3. Open the layout file activity_main.xml.
4. Add a Textview to display the application title.
5. Add Edit Text Fields to accept user input.
6. Add Buttons to perform actions such as submit or display data.
7. Create variables in MainActivity.java to ref UI components.
8. Implement onClick listener for the buttons to handle user interaction
9. Update the text view.
10. Run the application on the android emulator or physical device and observe how UI layer interacts with user inputs and updates the screen.

Output:

The application displays the title LAB5 - APP Architecture (UI Layer) and provides UI Components such as TextView, EditText and Buttons.

Result:

Thus, the Android application demonstrating the UI Layer in App architecture was successfully built and executed.


LAB 6
App Architecture - Persistence
This app demonstrates different persistence methods in Android architecture
Architecture Components
SharedPreferences
Simple key-value pairs for primitive data
Internal Storage
Files stored in app's private directory
Room Database
Structured data storage with SQLite
SharedPreferences
Simple key-value storage for primitive data
Enter your name
Manishaa
Save Name
Clear
Saved Name: Manishaa
Internal Storage
Files stored in app's private directory

Ex. No : 6

App architecture (persistence)

Aim :

To understand the persistence layer in Android app architecture.

Algorithm :

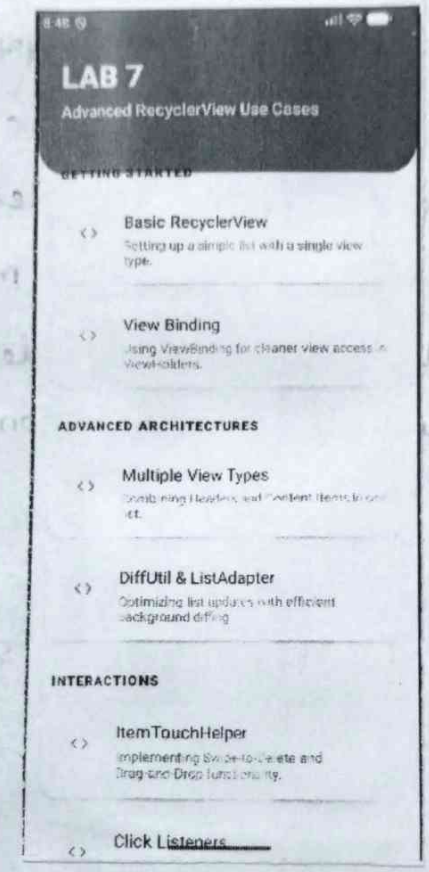
1. Open Android Studio and create new project.
2. Name the project and set package name.
3. Open the layout file and add textview to display.
4. Add Textview to display the title.
5. Add Edit Text fields to enter the data.
6. Add Buttons and create variables.
7. Implement a local persistence mechanism
8. Save button click listener code is written.
9. Build and run the application on android emulator.

Output :

The application displays LAB6 - App architecture (persistence)

Result.

Thus, the android application demonstrating the persistence layer in App architecture was successfully built and executed.


LAB 7
Advanced RecyclerView Use Cases
GETTING STARTED
Basic RecyclerView
Setting up a simple list with a single view type.
View Binding
Using ViewBinding for cleaner view access in ViewHolders.
ADVANCED ARCHITECTURES
Multiple View Types
DiffUtil & ListAdapter
Optimizing list updates with efficient background diffing
INTERACTIONS
ItemTouchHelper
Implementing Swipe-to-Delete and Drag-and-Drop functionality.
Click Listeners

Ex. No: 7

Advanced recycler view use cases

Aim:

To understand the advanced usage of Recycler View in Android.

Algorithm:

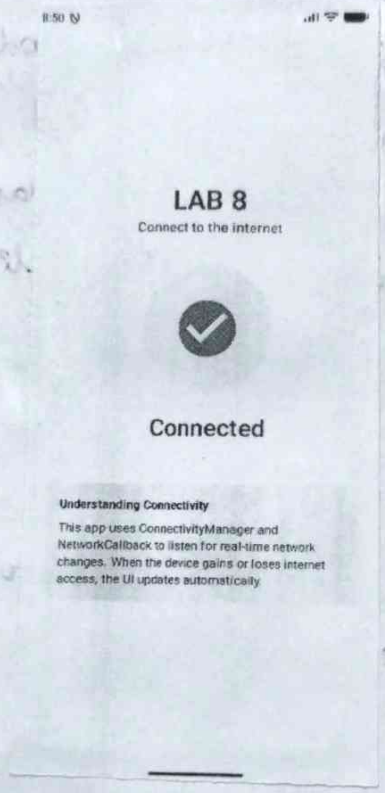
1. Create and Name the project & set package.
2. Add Recyclerview dependency.
3. Open activity_main.xml and add a Recyclerview.
4. Create a layout file and model class.
5. Implement ViewHolder to hold references
6. Bind the data from model class.
7. Implement advanced features like click listeners...
8. Build and run the application.

Output:

The application displays a scrollable list of items using Recyclerview.

Result:

Thus, the android application demonstrating advanced Recyclerview use cases successfully built and executed.


8:50
LAB 8
Connect to the internet
Connected
Understanding Connectivity
This app uses ConnectivityManager and NetworkCallback to listen for real-time network changes. When the device gains or loses internet access, the UI updates automatically.

Ex. No : 8

Connect to the Internet.

Aim:

To understand how to connect an Android application to the internet and retrieve data from an entire source using network connectivity

Algorithm:

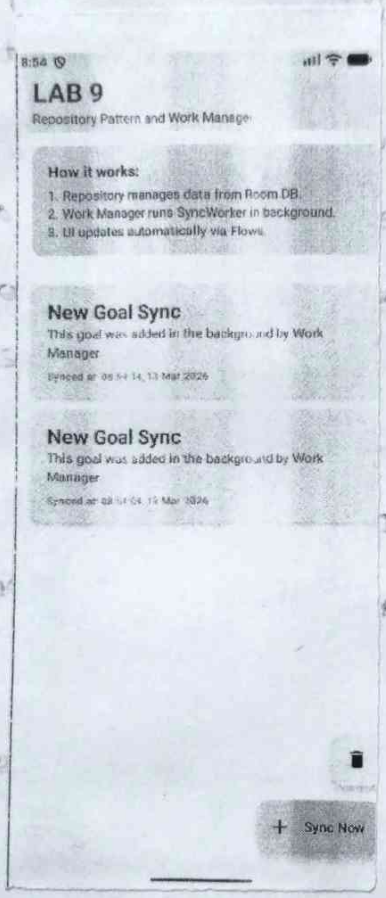
1. Name the project and set package.
2. Open the file AndroidManifest.xml.
3. Add the internet permission.
4. Add a Textview and add a button.
5. Declare variables for UI components.
6. Use a network library to request data from web URL.
7. Display the retrieved data in Textview.
8. Build and run the application on Emulator.

Output:

The application successfully connects to the internet.

Result:

Thus, the Android application demonstrating internet connectivity was successfully built and executed.


8:54
LAB 9
Repository Pattern and Work Manage
How it works:
1. Repository manages data from Room DB.
2. Work Manager runs SyncWorker in background.
3. UI updates automatically via Flows.
New Goal Sync
This goal was added in the background by Work Manager
Synced at: 08:54:14, 13 Mar 2026
New Goal Sync
This goal was added in the background by Work Manager
Synced at: 08:54:04, 13 Mar 2026
Sync Now

Ex. No : 9

Repository Pattern and Work Manager

Aim:

To understand the Repository pattern and work manager in Android app architecture for managing data operations.

Algorithm:

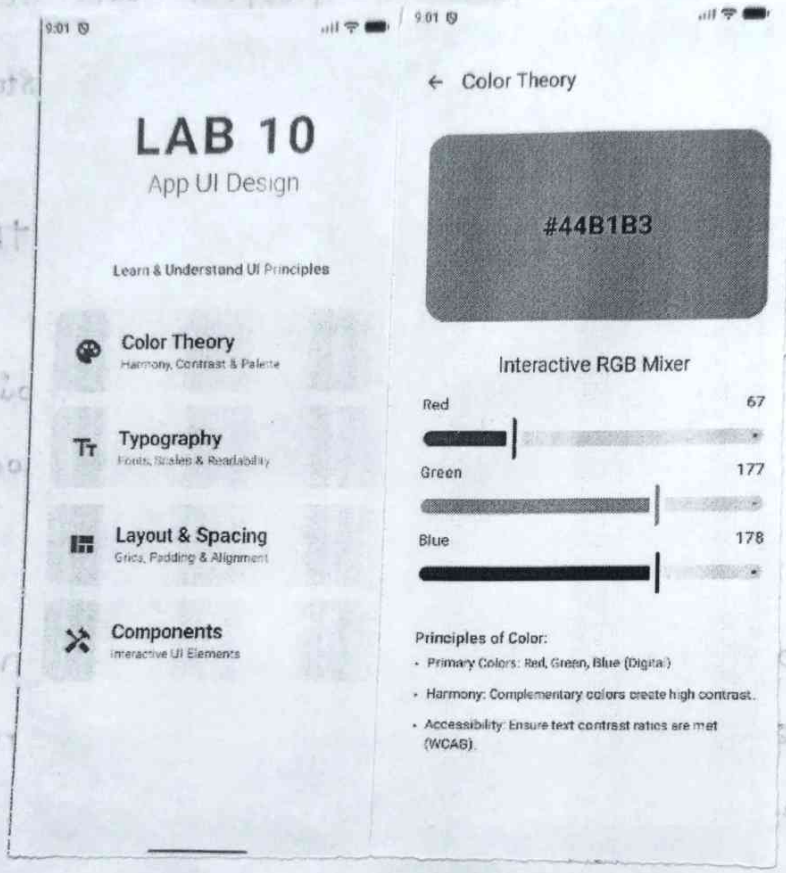
1. Create a new Android project and set package.
2. Add Work Manager and create a Repository class.
3. Implement functions in repository.
4. Create a worker class and override the doWork().
5. Initialize workManager in MainActivity.
6. Create a Work Request and use repository class.
7. Run the app and observe background.

Output:

The application demonstrates data management using Repository pattern and background task execution.

Result:

Thus, the Android application implementing the repository pattern was successfully built and executed.


9:01
LAB 10
App UI Design
Learn & Understand UI Principles
Color Theory
Harmony, Contrast & Palette
Typography
Fonts, Scales & Readability
Layout & Spacing
Grids, Padding & Alignment
Components
Interactive UI Elements
9:01
← Color Theory
#44B1B3
Interactive RGB Mixer
Red
67
Green
177
Blue
178
Principles of Color:
• Primary Colors: Red, Green, Blue (Digital)
• Harmony: Complementary colors create high contrast.
• Accessibility: Ensure text contrast ratios are met (WCAG).

Exc. No : 10

App - UI Design.

Aim:

To design and implement an attractive and user-friendly user interface for an android app.

Algorithm.

1. Open Android studio and create new project.
2. Name the project and set package.
3. Open the layout file.
4. Choose a suitable layout structure.
5. Add TextView and Edit Text fields, Buttons.
6. Apply styles, colors and margins.
7. Arrange UI components. and save layout.
8. Build and run the application.

Output:

The application displays a well-structured user interface with various UI components.

Result:

Thus, the android app demonstrating UI design was successfully built and executed.